

MongoDB y Beanie ODM



Objetivos de Aprendizaje

- Comprender qué es MongoDB y cuándo usarlo
- Aprender las operaciones CRUD básicas en MongoDB
- Introducir Beanie como ODM para Python
- Configurar una aplicación básica con MongoDB y Python



Introducción a MongoDB



¿Qué son las bases de datos NoSQL?

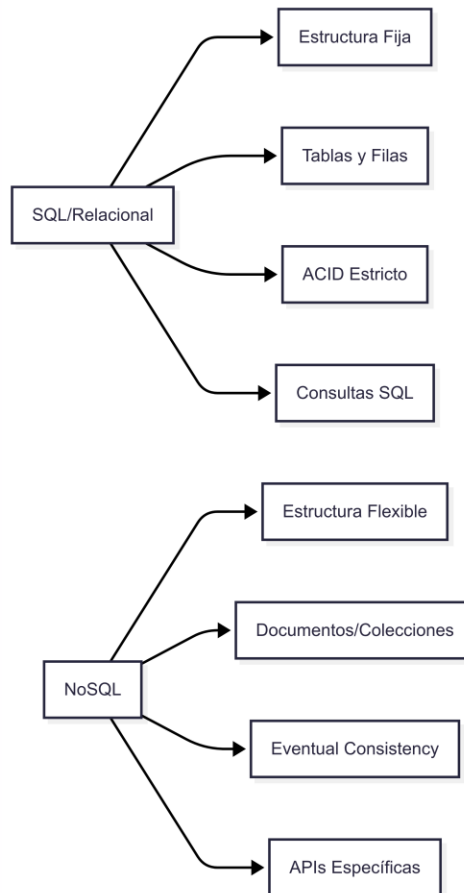
- NoSQL** = "**Not Only SQL**" - No solo SQL
- Alternativa a las bbdd relacionales tradicionales
- Diseñadas para manejar grandes volúmenes de datos
- Más flexibles en el esquema de datos

Tipos principales:

1. **Documentales** (MongoDB, CouchDB)
2. **Clave-Valor** (Redis, DynamoDB)
3. **Columnares** (Cassandra, HBase)
4. **Grafos** (Neo4j, Amazon Neptune)



SQL vs NoSQL: Diferencias Clave



- **SQL (Relacional)**

- Esquema fijo y estructurado
- Tablas con relaciones
- ACID (Atomicidad, Consistencia, Aislamiento, Durabilidad)

- **NoSQL:**

- Esquema flexible
- Documentos, clave-valor, grafos
- Escalabilidad horizontal
- APIs específicas por tecnología

¿Qué es MongoDB?

- Base de datos **documental** (almacena documentos JSON-like)
- **Open Source** y muy popular
- Diseñada para **escalabilidad** y **flexibilidad**
- Los datos se almacenan en formato **BSON** (Binary JSON)
- **Características principales**
 - Sin esquema fijo (schema-less)
 - Consultas ricas y expresivas
 - Indexación automática
 - Replicación y sharding integrados
 - Agregaciones poderosas



Terminología MongoDB vs SQL

SQL	MongoDB	Descripción
Base de datos	Base de datos	Contenedor de colecciones
Tabla	Colección	Grupo de documentos
Fila	Documento	Registro individual
Columna	Campo	Atributo de un documento
Índice	Índice	Mejora el rendimiento
Clave primaria	_id	Id único

```
1 Base de datos: tienda_online
2   | Colección: usuarios
3   |   | Documento: {
4   |   |   | _id: 1,
5   |   |   | nombre: "Juan",
6   |   |   | email: "juan@email.com"
7   |   |   | }
8   |   | Documento: {
9   |   |   | _id: 2,
10  |   |   | nombre: "Ana",
11  |   |   | edad: 25,
12  |   |   | ciudad: "Madrid"
13  |   |   | }
14  | Colección: productos
15  |   | Documento: {
16  |   |   | _id: 1,
17  |   |   | nombre: "Laptop",
18  |   |   | precio: 999,
19  |   |   | categoría: "electrónicos"
20  |   |   | }
21  |   | Documento: {
22  |   |   | _id: 2,
23  |   |   | nombre: "Mesa",
24  |   |   | precio: 150,
25  |   |   | material: "madera"
26  |   |   | }
```

Documentos BSON en MongoDB

```
1 {
2   "_id": ObjectId("507f1f77bcf86cd799439011"),
3   "nombre": "Juan Pérez",
4   "edad": 30,
5   "email": "juan@email.com",
6   "direccion": {
7     "calle": "Calle Mayor 123",
8     "ciudad": "Madrid",
9     "codigo_postal": "28001"
10  },
11  "hobbies": ["lectura", "natación", "programación"],
12  "fecha_registro": ISODate("2023-01-15T10:30:00Z"),
13  "activo": true
14 }
```

- Los documentos son como **objetos JSON** pero en formato binario (BSON)
- Cada documento tiene un **_id** **único** automático
- Los campos pueden ser de diferentes tipos
- Pueden contener **documentos anidados** y **arrays**

¿Cuándo Elegir MongoDB?



Usar MongoDB cuando:

- Los datos tiene **estructura variable**
- Trabajas con **JSON/documentos** frecuentemente
- Prototipado rápido y desarrollo ágil
- Datos semi-estructurados o no estructurados
- Aplicaciones web modernas con APIs JSON



No usar MongoDB cuando

- Necesitas **transacciones ACID complejas**
- Relaciones complejas entre entidades
- Reporting y análisis SQL complejos
- Presupuesto limitado para licencias



Operaciones CRUD en MongoDB



CRUD en MongoDB



```
1 // CREATE
2 insertOne() // Insertar un documento
3 insertMany() // Insertar múltiples documentos
4
5 // READ
6 find() // Buscar documentos
7 findOne() // Buscar un documento
8
9 // UPDATE
10 updateOne() // Actualizar un documento
11 updateMany() // Actualizar múltiples documentos
12
13 // DELETE
14 deleteOne() // Eliminar un documento
15 deleteMany() // Eliminar múltiples documentos
```



CREATE – Insertar Documentos

Insertar un documento

```
1 db.usuarios.insertOne({
2   nombre: "María García",
3   edad: 28,
4   email: "maria@email.com",
5   ciudad: "Barcelona"
6 })
```

Insertar múltiples documentos

```
1 db.usuarios.insertMany([
2   {
3     nombre: "Carlos López",
4     edad: 35,
5     email: "carlos@email.com"
6   },
7   {
8     nombre: "Ana Martín",
9     edad: 22,
10    email: "ana@email.com",
11    telefono: "123456789"
12  }
13 ])
```

READ – Consultar Documentos

```
1 // 1. Encontrar todos los documentos:
2 db.usuarios.find()
3
4 // 2. Encontrar con filtros:
5 // Por campo específico
6 db.usuarios.find({edad: 28})
7
8 // Mayor que
9 db.usuarios.find({edad: {$gt: 25}})
10
11 // Múltiples condiciones
12 db.usuarios.find({
13   edad: {$gte: 18},
14   ciudad: "Madrid"
15 })
```

```
1 // 3. Encontrar uno:
2 db.usuarios.findOne({email: "maria@email.com"})
3
4 // 4. Proyección (campos específicos):
5 db.usuarios.find(
6   {edad: {$gt: 25}},           // filtro
7   {nombre: 1, email: 1, _id: 0} // proyección
8 )
9
10 // Operadores de consulta comunes:
11 - `>`, `gte`, `lt`, `lte` (comparación)
12 - `eq`, `ne` (igualdad)
13 - `in`, `nin` (en array)
```

UPDATE – Actualizar Documentos

```
1 // 1. Actualizar un documento:
2 db.usuarios.updateOne(
3   {email: "maria@email.com"},      // filtro
4   {$set: {edad: 29, ciudad: "Madrid"}} // actualización
5 )
6
7 // 2. Actualizar múltiples documentos:
8 db.usuarios.updateMany(
9   {ciudad: "Barcelona"},
10  {$set: {pais: "España"}}
11 )
```

```
1 //3. Operadores de actualización:
2 // $set - Establecer valor
3 {$set: {edad: 30}}
4
5 // $inc - Incrementar
6 {$inc: {edad: 1}}
7
8 // $push - Añadir a array
9 {$push: {hobbies: "cocinar"}}
10
11 // $pull - Quitar de array
12 {$pull: {hobbies: "lectura"}}
13
14 // $unset - Eliminar campo
15 {$unset: {telefono: ""}}
16
17 // 4. Upsert (crear si no existe):
18 db.usuarios.updateOne(
19   {email: "nuevo@email.com"},
20   {$set: {nombre: "Usuario Nuevo"}},
21   {upsert: true}
22 )
```

DELETE – Eliminar Documentos

```
1 // 1. Eliminar un documento:
2 db.usuarios.deleteOne({email: "maria@email.com"})
3
4 // 2. Eliminar múltiples documentos:
5 db.usuarios.deleteMany({edad: {$lt: 18}})
6
7 //3. Eliminar todos los documentos:
8 db.usuarios.deleteMany({})
```



Precauciones importantes:

- Siempre usar filtros específicos
- Hacer backup antes de eliminar
- Considerar soft delete (marcar como eliminado)

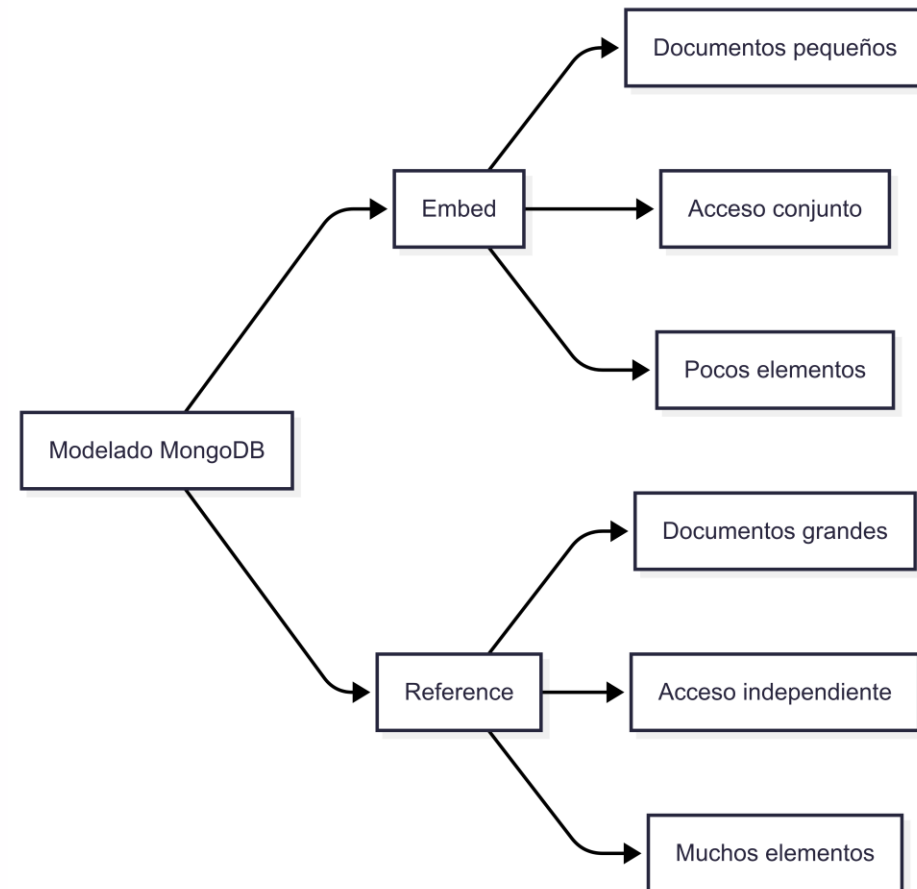
```
1 // En lugar de eliminar
2 db.usuarios.updateOne(
3   {email: "usuario@email.com"},
4   {$set: {eliminado: true, fecha_eliminacion: new Date()}}
5 )
6
7 // Consultar solo activos
8 db.usuarios.find({eliminado: {$ne: true}})
```

Modelado de Datos en MongoDB



Principios de Modelado en MongoDB

- Diferente al modelado relacional
- **Desnormalización** es común y aceptable
- Modelar según **patrones de acceso**
- Optimizar para **consultas frecuentes**
- **Principios clave**
 - **Embed vs Reference**
 - **One-to-Few** – Uno a pocos (embebido)
 - **One-to-Many** – Uno a muchos (referencia)
 - **Many-to-Many** – Muchos a muchos (array de referencias)



Documentos Embed vs Referencias

1. Documentos Embed

```
1 // Usuario con direcciones embebidas
2 {
3   _id: ObjectId("..."),
4   nombre: "Juan Pérez",
5   email: "juan@email.com",
6   direcciones: [
7     {
8       tipo: "casa",
9       calle: "Calle Mayor 123",
10      ciudad: "Madrid",
11      codigo_postal: "28001"
12    },
13    {
14      tipo: "trabajo",
15      calle: "Av. Castellana 200",
16      ciudad: "Madrid",
17      codigo_postal: "28046"
18    }
19  ]
20 }
```

2. Documentos Referencia

```
1 // Usuario
2 {
3   _id: ObjectId("user123"),
4   nombre: "Juan Pérez",
5   email: "juan@email.com"
6 }
7
8 // Pedidos (referenciando usuario)
9 {
10   _id: ObjectId("order456"),
11   usuario_id: ObjectId("user123"),
12   productos: [...],
13   total: 150.00,
14   fecha: ISODate("2023-01-15")
15 }
```

Patrones de Modelado MongoDB 1

1. One-to-Few

```
1 // Blog post con comentarios
2 {
3   _id: ObjectId("..."),
4   titulo: "Mi primer post",
5   contenido: "...",
6   comentarios: [
7     {
8       autor: "Ana",
9       texto: "Excelente post!",
10      fecha: ISODate("2023-01-15")
11    },
12    {
13      autor: "Carlos",
14      texto: "Muy útil, gracias",
15      fecha: ISODate("2023-01-16")
16    }
17  ]
18 }
```

2. One-to-Many

```
1 // Categoría
2 {_id: ObjectId("cat1"), nombre: "Electrónicos"}
3
4 // Productos (muchos productos por categoría)
5 {
6   _id: ObjectId("prod1"),
7   nombre: "Laptop",
8   categoria_id: ObjectId("cat1")
9 }
10 {
11   _id: ObjectId("prod2"),
12   nombre: "Mouse",
13   categoria_id: ObjectId("cat1")
14 }
```

Patrones de Modelado MongoDB 2

3. Many-to-Many

```
1 // Estudiante
2 {
3   _id: ObjectId("student1"),
4   nombre: "María",
5   cursos: [ObjectId("curso1"), ObjectId("curso2")]
6 }
7
8 // Curso
9 {
10  _id: ObjectId("curso1"),
11  nombre: "Python Básico",
12  estudiantes: [ObjectId("student1"), ObjectId("student2")]
13 }
```

Buenas Prácticas en MongoDB

- **Hacer:**

- Modelar según patrones de consulta
- Desnormalizar cuando sea necesario
- Usar documentos embebidos para datos que se consultan juntos
- Indexar campos de consulta frecuente
- Limitar el tamaño de documentos (16MB máximo)

- **Evitar:**

- Arrays que crecen indefinidamente
- Documentos muy profundos (más de 100 niveles)
- Referencias circulares
- Joins complejos



Reglas de Oro

1. Datos a los que se accede juntos, se almacenan juntos
2. Optimizar para el caso de uso más común
3. Referenciar es normal, no tengas miedo de hacerlo

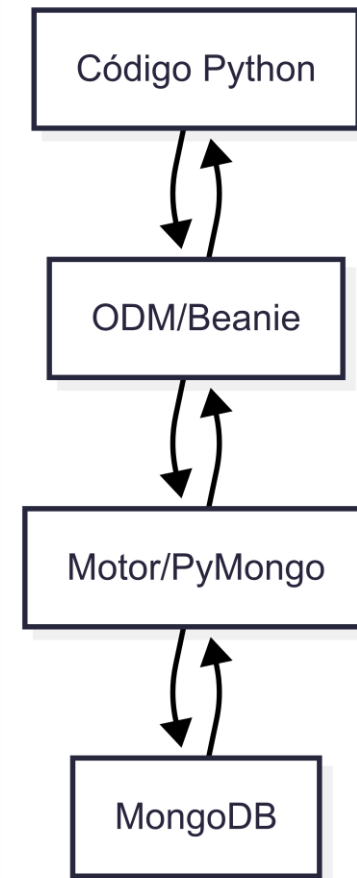


Introducción a Beanie ODM



¿Qué es un ODM?

- ORM = Object Relational Mapper
<-> Mapea objetos con tablas relacionales
- ODM = Object Document Mapper
<-> Mapea objetos con documentos NoSQL
- Propósito de un ODM
 - Abstrae la interacción con la base de datos
 - Proporciona validación de datos
 - Facilita el modelado orientado a objetos
 - Simplifica consultas complejas



Beanie ODM para Python

- Es un ODM **asíncrono** basado en **Pydantic** y **Motor**
- Diseñado para uso en aplicaciones modernas como **FastAPI**
- **Type hints** y validación automática
- Sintaxis simple y pythónica
- **Características principales**
 - Validación automática con Pydantic
 - Soporte completo para async/await
 - Migración automática de esquemas
 - Agregaciones complejas simplificadas
 - Integración nativa con FastAPI



Beanie: Pros y Contras



Ventajas

- **Type Safety:** Tipado estático on Python type hints
- **Validación automática:** Basado en Pydantic models
- **Asíncrono:** Perfecto para aplicaciones web modernos
- **Integración con FastAPI**
- **Sintaxis simple:** Fácil de aprender y usar



Desventajas

- **Relativamente nuevo:** Menor comunidad vs alternativas
- **Python 3.7+:** Requiere versiones modernas de Python
- **Curva de aprendizaje:** Necesitas conocer async/await
- **Documentación algo limitada**
- **Performance overhead:** Abstracción añade algo de latencia.



Workflow Típico con Beanie

- 1. Instalación y configuración**
- 2. Definir modelos**
- 3. Configurar conexión**
- 4. Operaciones CRUD**
- 5. Integración con FastAPI usando lifespan**



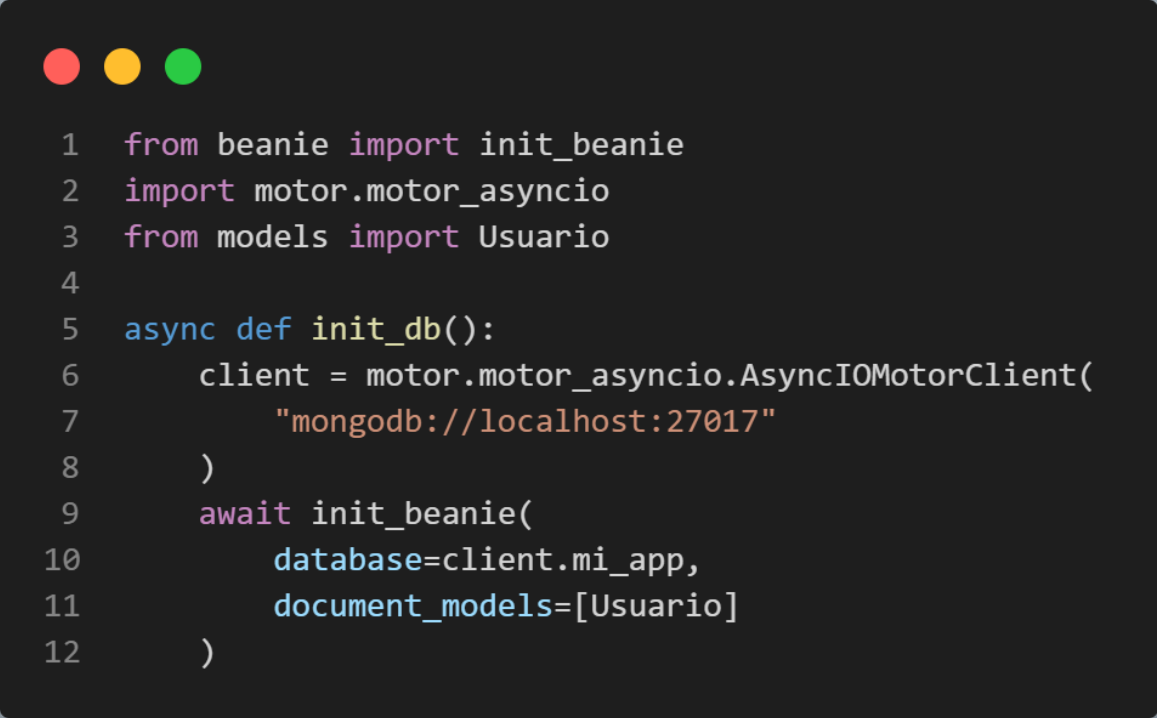
Ejemplo de modelo



```
1  from beanie import Document
2  from pydantic import EmailStr
3  from typing import Optional
4  from datetime import datetime
5
6  class Usuario(Document):
7      nombre: str
8      email: EmailStr
9      edad: Optional[int] = None
10     fecha_registro: datetime = datetime.now()
11     activo: bool = True
12
13     class Settings:
14         name = "usuarios"
```



Ejemplo de database.py



```
1 from beanie import init_beanie
2 import motor.motor_asyncio
3 from models import Usuario
4
5 async def init_db():
6     client = motor.motor_asyncio.AsyncIOMotorClient(
7         "mongodb://localhost:27017"
8     )
9     await init_beanie(
10         database=client.mi_app,
11         document_models=[Usuario]
12     )
```

Ejemplo de CRUD simplificado



```
1 # Create
2 await MiModelo(...).insert()
3
4 # Read
5 await MiModelo.find_one(MiModelo.campo1 == "valor")
6
7 # Update
8 await objeto.set({MiModelo.campo2: nuevo_valor})
9
10 # Delete
11 await objeto.delete()
```

Otros métodos que podemos usar es `.create()` desde el objeto para crearlo directamente, en vez de `insert` y `.get()` en vez de `find_one` porque le podemos pasar directamente el id.

Ejemplo de integración con lifespan

```
1 from contextlib import asynccontextmanager
2
3 @asynccontextmanager
4 async def lifespan(app: FastAPI):
5     """
6     Context manager para inicializar la conexión al lanzar el servidor
7     y cerrarla cuando se acaba ese proceso.
8     """
9     await init_db()
10    yield
11
12 app = FastAPI(
13     lifespan=lifespan,
14     title="School API",
15     description="A simple API to manage students in a school.",
16     version="1.0.0",
17 )
```

kingscorner